Ders 5 Genişletilmiş Notu: Fonksiyonel Çözümleme: Veri Akış Diyagramları, Süreç Tanım Formları, Karar Tabloları ve Veri Sözlüğü

Ders 5 Genişletilmiş Notu: Fonksiyonel Çözümleme: Veri Akış Diyagramları, Süreç Tanım Formları, Karar Tabloları ve Veri Sözlüğü

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Sistem Analizinin İkinci Aşaması: Çözümleme
- 2. Gereksinim Analizinin Amaçları
- 3. Analiz Modeli: Çember ve Merkezindeki Veri Sözlüğü
- 4. Veri Akış Diyagramı Nedir?
- 5. Veri Akış Diyagramının Dört Öğesi
- 6. Semboller: Gane-Sarson ve DeMarco-Yourdon
- 7. Çizim Kuralları ve Sık Yapılan Hatalar
- 8. Veri Akış Diyagramı Düzeyleri
- 9. Hastane Fatura Sistemi: Taslak ve Birinci Düzey
- 10. İkinci Düzey ve Düzeyler Arası Denge
- 11. İki Kısa Düzey Örneği
- 12. Mantıksal ve Fiziksel Veri Akış Diyagramı
- 13. Süreç Tanımlama Formu
- 14. Sözde Kod
- 15. Karar Tablosu
- 16. Sonlu Durum Makineleri ve Eş Zamanlı Sistemler
- 17. Karar Ağacı
- 18. Veri Sözlüğü
- 19. Erişilebilirlik Hangi Fizibilitenin Konusu?

1. Sistem Analizinin İkinci Aşaması: Çözümleme

Geçen hafta iş analizini gördük: İş analizi dokümanında neler bulunur, var olan sistem incelenerek ve mevcut çalışanlardan bilgi toplanarak gereksinimler nasıl ortaya çıkarılır. Bu hafta sistem analizinin ikinci aşamasına, yani çözümlemeye geçiyoruz. Hoca bu aşamayı sürecin en önemli ve üzerinde en çok durulacak adımlarından biri olarak tanıttı.

Çözümleme, iş analizinde bulunan gereksinimlerin (örneğin kullanım senaryolarıyla açıklanan iş gereksinimlerinin) daha teknik bir hale dönüştürülmesidir. Amaç, gereksinimlerin bir yazılım ya da yazılım ailesi olarak çalışabilecek bir sistem çözümüne doğru ilerlemesini sağlamaktır. Bu yüzden hoca bu safhayı “teknik dile dönüştürmenin ilk adımı” diye de adlandırdı.

1.1 İki Çözümleme Yöntemi

Bu aşama iki hafta boyunca işlenecek, çünkü iki temel çözümleme yöntemi vardır:

- **Fonksiyonel (işlevsel) çözümleme:** Geleneksel yaklaşımla, işlev temelli bir soyutlama ve bölükleme yapmak istediğinizde kullanılır. "İşlev" ve "fonksiyon" aynı şeyi ifade eden kelimelerdir. Bu dersin konusu budur.
- **Nesneye dayalı çözümleme:** Nesneye dayalı yazılım geliştirilecekse kullanılır. Aday sınıflar (kavram uzayı), ileride nesne olarak çalışacak atomik birimler, bunların özellikleri ve davranışları ortaya çıkarılır ve uygun diyagramlarla gösterilir. Bu, gelecek haftanın konusudur.

Bu derste şu sorulara bakılacak: Fonksiyonel çözümleme nedir, ne değildir, hangi diyagramlar çizilir, çizerken nelere dikkat edilir, neyi neden çiziyoruz?

1.2 Bu Safhanın Çıktısı: SRS

Her safhada olduğu gibi burada da bütün paydaşlara yapılanları aktaran bir çıktı üretmemiz beklenir. Bu çıktının adı SRS'dir (Software Requirement Specification, yazılım gereksinim spesifikasyonu). "Yazılım gereksinimlerinin tam tanımı" diye de adlandırılabilir. İş gereksinimleri bu dokümanın ilk bölümü olarak eklenebilir. Böylece analiz safhasında yapılan her şey A'dan Z'ye tek bir dokümanda raporlanıp ilgili taraflara aktarılabilir.

2. Gereksinim Analizinin Amaçları

Hoca, analizi neden yaptığımızı hatırlattı. Bir probleme çözüm üretecek bir sistem geliştirmek istiyoruz. Mevcut durumda işlerin yapılış şekilleri, çıktıların üretilme şekilleri ve iş akışları var. Yeni sistemde hem bunlar hem de belki yeni istekler ve ihtiyaçlar olacak. Bunları ortaya çıkarmak ve formal olarak tanımlamak için analiz yapıyoruz. Gereksinim analizinin üç amacı şöyle özetlenebilir:

- **Problemi anlamak:** İhtiyaçların neler olduğunu ve nasıl işlediklerini ortaya koymak.
- **Tasarıma temel oluşturmak:** Tasarım, bir çözüm önerisidir. Problemin parçaları ve ihtiyaçlar tanımlandıktan sonra bu tanımlardan yola çıkılarak bir çözüm önerisi geliştirilir. Çözüm adayı üretebilmek için önce problemi anlamak gerekir.
- **Geçerleme için materyal üretmek:** Sistem bittiğinde şu sorular sorulacak: Gerçekten doğru bir ürün gerçekleştirdik mi? Ürün beklentileri, ihtiyaçları ve gereksinimleri karşılıyor mu? Bunu onaylamak için gerekli unsurlar da gereksinim analizinde belirlenir.

Gereksinim analizini yaparken de sistem geliştirme sürecinin tamamında olduğu gibi, daha önce defalarca denenmiş ve iyi çalıştığı bilinen yollardan yürümeyi tercih ederiz. Bu sistematik yollara model denir. Analiz modeli, neyin iyi neyin kötü olduğu bilindiği için iyi adımların ve akışların bir araya getirilerek kullanıldığı süreçtir.

3. Analiz Modeli: Çember ve Merkezindeki Veri Sözlüğü

Hoca analiz modelini üç dilime ayrılmış bir çember olarak gösterdi. Her dilim bir tanım çalışmasıdır, merkezde ise bunları birbirine bağlayan veri sözlüğü bulunur.

Çemberin parçası	Sorduğu soru	Kullanılan araç
Veri nesne tanımı	Veriler neler, yapıları ve ilişkileri ne?	Varlık ilişki diyagramı
Süreç tanımı (süreç spesifikasyonu)	Hangi işler var, nasıl akıyor, kimlerle etkileşiyor?	Veri akış diyagramı
Kontrol tanımı (kontrol spesifikasyonu)	Hangi olayla hangi duruma geçilir?	Durum geçiş diyagramları, sonlu durum makineleri
Veri sözlüğü (merkez)	Modeldeki her şeyin tanımı ne?	Sözlük girdileri

3.1 Veri Nesne Tanımı

Sistem, girdi-işlem-çıktı (input, process, output) zinciriyle çalışır. Bilgi üretmek için veriye ihtiyaç vardır. Bu yüzden kullanılan verinin ne olduğunu, veri yapılarını ve hangi verinin hangi

veriyle nasıl bir bağlantısı olduğunu bilmek ve tanımlamak gerekir. Bu çalışma varlık ilişki diyagramı tarzında bir çözümlemeyle yapılır. Analiz safhasında başlar, tasarım safhasında neticelenir ve o zaman sistemin veri tasarımı adını alır.

3.2 Süreç Tanımı

Gereksinim analizinin belki de en önemli ve en uzun süren işi süreç tanımıdır. Sistemde hangi işler ve sorumluluklar var, bunlar nasıl akıyor, aralarında nasıl ilişkiler var, hangi paydaşlarla etkileşime girmek zorundalar? Örneğin bir süreç çalışmak için bir kullanıcıdan bilgi almak, birine rapor üretmek ve bazı verilere bir yerden erişmek zorunda olabilir. Sistemin bütün işlevselliği burada gösterilip tanımlanır.

Fonksiyonel çözümlemede bu dilim için veri akış diyagramları (data flow diagram) kullanılır. Veriler kaynaktan işleve, işlevden işleve, işlevden veri saklama birimlerine akar. Hoca işlevleri bir değirmene benzetti: Veri orada öğütülür, başka bir hale getirilir ve başka modüllerin ya da servislerin kullanımına sunulur.

3.3 Kontrol Tanımı

Sistem bazen dışarıdan gelen olaylara tepki vererek sonuç üretmek zorundadır, yani olay güdümlü çalışır. Bu çok ön plandaysa kontrol spesifikasyonunun net olarak tanımlanması gerekir. Burada durum geçiş diyagramları ve daha karmaşık durum diyagramları ya da sonlu durum makineleri kullanılır.

Hocanın örnekleri şunlardı: Kullanıcı kartını takarsa ne olur? Kartını takıp aynı anda bir tuşa basarsa ne olur? Barkod okuyucudan bir şey okunduğu anda dış sistemden bir sipariş isteği gelirse ne olur? Bir olay bir zaman bile olabilir: Gece 12.00'de alarm çalıp sisteme "gün döndü" diye bir tetikleme gelebilir.

Böyle bir etki geldiğinde sistem A durumunda kalamaz. Belli işleri yapıp belli verileri kullanarak sonuç üretir ve B durumuna geçer. Her geçiş yeni bir sorumluluk demektir.

> **Ders sonrası not:** Altyazıdaki "stake çarklar" ve "stake diagramlar" ifadeleri, kısa nottaki düzeltmeye göre büyük olasılıkla "statechart" ve "state diagram" (durum diyagramı) terimleridir.

3.4 Dilimler Birbirini Etkiler

Çemberin dilimleri birbirinden bağımsız değildir. Bir durum geçişi süreç spesifikasyonuna yeni işlemler ekletir. Beraberinde yeni veriler kullanılıyorsa veri nesne tanımı da güncellenir. Tersine, gereksinimlerde bir değişiklik olur da yeni bir işlev eklenir ya da bir işlevin akışı değişirse bu diğer dilimleri de etkileyebilir. Üç tanım düzgün yapılabilirse analiz gerçekleştirilmiş ve modeli ortaya konmuş olur.

3.5 Veri Sözlüğü

Üç pasta dilimini bir araya getirmek gerekir. Hocanın benzetmesiyle, arabaya tekerleği takıp bijonları sıkmak gibi, ortadan bir cıvata bağlamak lazımdır ki parçalar dağılmasın. Yarın başka biri bu "tekerleğin" başına geçtiğinde "bunlar neymiş" diye tökezlememelidir.

Veri sözlüğü bir külliyat, bir kütüphanedir. Analiz modelindeki her şey, en ufak veri tipinden en karmaşık süreç ya da durum tanımına kadar, burada bir kayıt olarak durur. Üç dilimde ne üretilirse sözlüğe girilir. Bir değişiklik yapılırsa sözlükteki ilgili bütün maddeler gözden geçirilip güncellenir.

3.6 Herkes Bu Kadar Ayrıntılı Yapıyor mu?

Bu iş el oyalayıcı, yorucu ve dikkat gerektiren bir iştir. Hoca soruyu kendisi sordu ve cevapladı: Hayır, herkes yapmıyor. Günlük hayatta buna ayrılacak zaman pek olmuyor. Özellikle çevik süreçlerle birlikte iş hızlandı ve çok uzun, ayrıntılı diyagramlar pek beklenmiyor. Ancak görev açısından kritik (mission critical) uygulamalar vardır. Uzaya roket gönderiyorsanız

ve onun kontrol yazılımını geliştiriyorsanız bunların hepsini en ince noktasına kadar yapmak zorundasınız.

> **Hocanın vurgusu:** Burada anlatılanların büyük ihtimalle %60-70'ini ileride kullanmayacaksınız, buradan edindiğiniz genel tecrübeyle ilerleyeceksiniz. Ama bazı projelerde bu kadar ayrıntıya inmek zorunda kalabilirsiniz. O gün "ben bunu bilmiyorum" dememek için bütün tanımları, kitabına uygun biçimde bilmeniz gerekir.

3.7 Bu Dersin Kapsamı

Bu ders süreç tanımına ve veri sözlüğüne odaklanır. Çemberin kontrol dilimi ders kapsamında neredeyse hiç işlenmeyecek. Hocanın deyimiyle burası "don't care", yani yokmuş gibi davranılan kısımdır. Sistemleri kombinatoryal kabul ediyoruz: Durum geçişlerinin çok yaşanmadığını, olay tetiklemesinin az ya da çok temel seviyede olduğunu varsayıyoruz. Veri modellemesi ise ilerleyen haftalarda, hocanın hatırladığına göre 10. hafta civarında, bir haftalık ders olarak işlenecek.

Özetle çemberin bizim için önemli kısmı üçte ikisidir, kalan üçte biri şimdilik görmezden gelinecek.

4. Veri Akış Diyagramı Nedir?

Modelleme bir çeşit ifade etmedir. Analiz modelindeki süreçleri ifade etmek için görsel bir dil kullanmak isteriz. Sebep, kolay anlaşılır olmasıdır. Yazılımcı, mimar ya da güvenlik uzmanı gibi teknik bir kişi baktığında kendi özelinde bir çıkarım yapabilmeli. İşten hiç anlamayan bir fabrika sahibi de baktığında "evet, biz bu bilgileri alıyoruz, bu taraflarla konuşmak zorundayız" diyebilmelidir. İki tarafın da anlayacağı basitlikte bir dil gerekir.

Veri akış diyagramı adı üstünde, bazı verilerin bir yerlerden başlayıp bir yerlere aktığını gösterir. Bütün sistemler nihayetinde girdi, işlem ve çıktıdan oluşur. Girdiler gelir, işlenir, süreçler arasında paylaşılır, birinin çıktısını öbürü girdi olarak alır. Bir yerlerde depolanır ya da depodan okunur, işlenir, bir yere yazılır ya da bir tarafa gönderilir. Bu akışları tanımlarsak iş analizindeki sorumluluklarımızı yerine getirmek için hangi parçalara, hangi verilere, hangi veri kaynaklarına ve hangi işlevlere ihtiyacımız olduğunu kutu kutu gösterebiliriz.

Tanım: Sistemdeki varlıkları, süreçleri, veri depolarını ve bunların arasındaki veri akışlarını bir arada gösteren görsel ifade şekline veri akış diyagramı (VAD) denir. Hangi veri nereden gelip nereye gidiyor ve neden, hepsi burada görülür. Varlıklar geçen hafta aktör olarak modellediğimiz kavramlara karşılık gelir.

4.1 Öğrenci Kaydı Örneği

Hoca slaytta bir öğrenci varlığının bir süreçle veri akışı kurduğunu gösterdi. Teknik bir kişi bunu şöyle okur: Sistemin etkileşime geçtiği bir öğrenci varlığı var. "Yeni öğrenci kaydı" sürecini işletebilmek için bu varlıktan veri alınacak. Alınacak verinin adı okun üzerine yazılır, örneğin kimlik verileri. Süreç sonunda belli bilgiler belli bir formatta üretilip bir yere gönderilir ya da depolanır.

4.2 Diyagramın Genel Çizim Alışkanlıkları

Hoca kuralların ayrıntısına geçmeden önce üç temel alışkanlığı anlattı:

- **Tekrarlanan varlık:** Bir sistemde birden fazla varlık olur ve bir varlık birçok süreçle ilişkilidir. Bir süre sonra oklar birbirinin üzerinden geçmeye başlar. Çakışmayı önlemek için aynı varlık diyagramda birkaç kez çizilebilir. Hepsi aynı varlıktır, tekrar ettiği köşesine bir çentik atılarak gösterilir.
- **Tek yönlü ok:** Veri akar, bu yüzden oklar tek yönlüdür. Bir varlık sürece girdi veriyorsa süreç aynı bilgiyi geri vermez. Ters yönde bir veri akışı varsa ayrı bir okla gösterilir.
- **Tek kaynak, tek hedef:** Her okun bir kaynağı ve bir hedefi olması beklenir. Aynı bilgi iki farklı sürece giriyorsa ok bir yerden sonra makasla ortadan ikiye bölünür gibi çizilir: Bir yere kadar tek ok, sonra çift ok.

Ayrıca işlemler hiyerarşiye uygun olarak numaralandırılır: 1, 1.1, 1.2 gibi.

5. Veri Akış Diyagramının Dört Öğesi

5.1 Varlıklar

Varlıklar, sistemle etkileşime geçen gerçek kişiler, kurumlar, birimler, farklı sistemler ya da otomatik araçlardır, yani aktörlerimiz gibidir. Birincil aktörlere karşılık gelirler, destek aktörleri de olabilirler. Sisteme veri sağlarlar ya da üretilen çıktıları kullanmak üzere alırlar. Hoca bunlardan "entity, source ya da sink" diye de söz etti.

5.2 Veri Akışları

Veri akışı, sistemde bir kaynaktan yola çıkıp bir hedefe doğru hareket eden veridir. Soyut bir kavramdır. Müşteri numarası, barkod numarası, öğrenci numarası ya da T.C. kimlik numarası gibi tek bir atomik değer olabilir. Grafikleri ve liste dökümleri olan kocaman bir rapor da olabilir. Hangisi olursa olsun diyagramda hepsi tek bir oktur.

5.3 Süreçler

Süreçler (proses, işlem), sistemde yerine getirmemiz gereken sorumluluklardır: Bir işi gerçekleştirmek amacıyla yürütülen etkinlikler ya da fonksiyonlar. Emir kipindeki yüklemlerle adlandırılırlar, örneğin "Randevu al". Bu süreç hasta, banko görevlisi ya da sekreter gibi varlıklardan bilgi alabilir, onlara bilgi verebilir, veri depolarından okuyup yazabilir.

5.4 Veri Depoları

Veri depoları verinin kalıcı olarak saklandığı alanlardır. Dosya, disk, klasör, veri tabanı yönetim sistemindeki bir tablo, form, çıktı, rapor, kare kod olabilir. Bu da soyut bir kavramdır.

> **Hocanın vurgusu:** Veri deposu deyince aklımıza illa veri tabanı tablosu gelmesin. Verinin kalıcı olarak durduğu herhangi bir yer veri deposu olabilir.

6. Semboller: Gane-Sarson ve DeMarco-Yourdon

Literatürde birbirinden çok da farklı olmayan iki çizim şekli vardır: DeMarco-Yourdon sembolleri ve Gane-Sarson sembolleri. Ders kapsamında ve Yıldız Teknik Üniversitesi'ndeki alışkanlıkla Gane-Sarson sembolleri kullanılır.

> **Ders sonrası not:** Altyazıda "Demarco-Jurdon" ve "Gain ve Sarson" diye geçiyor. Kısa nottaki yazıma göre doğru adlar büyük olasılıkla DeMarco-Yourdon ve Gane-Sarson'dur.

Öğe	Gane-Sarson gösterimi	Üzerine yazılan
Süreç	Kenarları yuvarlatılmış kutu, tepede ayrı numara bölümü	Hiyerarşik numara ve emir kipinde ad
Veri akışı	Ok	Veri akışının adı (isim)
Veri deposu	Dikdörtgen, kenarında numara	Depo numarası ve ad
Varlık	Dikdörtgen	Varlığın adı

Her öğenin ayrıntısı şöyledir:

- **Süreç:** Ad sığsın diye kutu genişletilebilir. Tepedeki bölümde hiyerarşik numara bulunur. Örneğin 1 "kayıt işlemleri" gibi genel bir işlevse, içine girildiğinde "yeni öğrenci kaydı", "af ile gelen öğrenci kaydı" ve "yatay geçiş öğrenci kaydı" gibi alt işlemler 1.1, 1.2, 1.3 numaralarını alır.
- **Veri akışı:** Okun üzerinde veri akışının adı bulunur. Basit ya da kompozit fark etmez, hepsi veri olduğu için isim olmak zorundadır. Fiil içermemesi beklenir.
- **Veri deposu:** Farklı tablo, form ya da dosyaların birbirinden ayrı olduğunu göstermek için süreçlerde olduğu gibi numaralandırılır: depo 1, depo 2.
- **Varlık:** Basit bir dikdörtgendir, içine adı yazılır.

Slayttaki örnekte süreç “Profil bilgileri gir” gibi etken ve emir kipinde adlandırılmıştı. Okun üzerinde ise “Doğrula” yazıyordu. Bu kasıtlı olarak yanlış bir örnektir, çünkü “doğrula” bir fiildir. Okun üzerinde “doğrulanmış hasta bilgisi” ya da “doğrulanmış hasta verisi” gibi bir isim olmalıdır. Hastaların kimliklerinin tutulduğu tablonun adı “hasta kimlik” ya da “hasta kimlik tablosu” olabilir. Dış varlık ise hastadır.

7. Çizim Kuralları ve Sık Yapılan Hatalar

Bir veri akış diyagramına bakmadan önce çizim kurallarını bilmek gerekir. Hepsinin altında aynı mantık yatar: Her işlev için girdi, işlem ve çıktının olduğunu göstermek ve mantıklı bir veri akışı tanımlamak. Fonksiyonel çözümleme tam olarak budur: Sistemi, gereksinimleri yerine getirmek için gereken işlevlere, işlemlere ve süreçlere parçalamak.

7.1 Girdisi Olmayan Süreç

İşlemler işlemek için girdiye ihtiyaç duyar. Bir şey almadan hiçbir çıktı üretilemez. Yalnızca çıkışı olan bir işlem mantıklı değildir. Diyagramda böyle bir şey görüyorsanız büyük ihtimalle hata vardır, unutulmuş bir varlık, veri deposu ya da gösterilmemiş bir veri akışı söz konusudur.

7.2 Çıktısı Olmayan Süreç

Her şeyi yutan bir işlem de olamaz. İşlemin yalnızca girdisi olup çıktısı olmaması doğru bir çizim değildir. Böyle bir şey varsa büyük ihtimalle çıktı unutulmuştur. Beklenen akış her zaman girdi, işlem, çıktı biçimindedir.

> **Ek bilgi (derste anlatılmadı):** Altyazıda bu iki hata için özel bir ad geçmiyor. Kısa nota göre literatürde girdisiz sürece “mucize”, çıktısız sürece “kara delik” denir. İngilizce kaynaklarda karşılıkları büyük olasılıkla “miracle” ve “black hole” terimleridir.

7.3 Çıktıyı Üretmeye Yetmeyen Girdi

Girdi olması yetmez, girdinin beklenen çıktıyı üretecek kadar yeterli olması beklenir. Hocanın örneği maaş hesaplama süreciydi. Diyagramda “çalışma dosyası” adlı bir veri deposundan “çalışma saatleri” bilgisi alınıyor. Bu dosyada örneğin şu kayıt 8 saat, bu kayıt 17 saat, bu kayıt 1 saat, bu kayıt 0 saat çalıştı gibi bilgiler var. Peki yalnız saatlerle maaş nasıl hesaplanacak? Burada mantık eksiği vardır. Şu girdiler de gerekir:

- **Çalışan dosyası:** Çalışanların kim olduğu ve görevi. Örneğin bir çalışan müdür, bir başkası depo elemanı ya da kurye.
- **Ücret tanımları:** Göreve göre saat ücreti, birim ücret ya da katsayı. Bu bilgi ayrı bir tanım tablosunda olabilir.

Bunlar alındıktan sonra saat ve katsayıdan maaş hesaplanır. Gerekirse vergi hesabı da katılarak ödenecek brüt ve net tutarlar bulunur. Müşteri diyagrama bakıp “saatleri aldın, peki bunlar kim, nereden biliyorsun?” diye sorabilir.

> **Hocanın vurgusu:** “Aslında onu biliyorum, kalabalık olmasın diye çizmedim” demek olmaz. Analiz modelinde bu işlem için hangi girdilerin sağlanacağını göstererek niyetinizi de ortaya koyarsınız. İstenen çıktıyı üretebilecek kadar bilgiyi aldığınızı veri akışlarıyla göstermeniz beklenir.

7.4 Süreçten Geçmeyen Akış

Veriler herhangi bir işleme girmeden bir yerden bir yere akamaz. Bir veri deposundan başka bir veri deposuna doğrudan ok çizmek, verinin dosyadan veri tabanı tablosuna “kendi kendine şakır şakır” aktığı anlamına gelir. Veri işlenmek üzere akar. Doğrusu şudur: Veri deposundan bir ok bir sürece girer, süreç sonucunda bir şey üretilir, o da başka bir veri deposuna yazılır. Maaş örneğinde maaş hesaplama sürecinden çıkan maaş bilgilerinin muhasebe ya da vergi için ayrı bir veri deposuna kaydedilmesi böyledir.

```
Yanlış:  [Depo 1] --------------------> [Depo 2]
Doğru:   [Depo 1] --> (Süreç) --> [Depo 2]
```

Aynı kural varlıklarla veri depoları arasında da geçerlidir. Varlıktan depoya doğrudan ok çizilmişse bu, o kişinin dosyaya doğrudan yazı yazdığı ya da tabloya doğrudan veri girdiği anlamına gelir. Varlıktan gelen veri akışı önce bir işleme girmelidir. Sonra nereye gidecekse (başka varlık, başka işlem, başka veri deposu) o işlemin sonucu olarak gider.

7.5 İki Yönlü Ok

Akışlar iki oklu olamaz. Bir işlem ile veri deposu arasındaki iki yönlü ok, işlemin depodan veri alıp aynı veriyi geri bastığı anlamına gelir. O zaman veri neden alınıyor? Doğrusu, okunan veriden farklı, işlenmiş bir verinin çıktı olarak üretilmesidir: Depodan A veri akışı alınır, işlemde işlenir, depoya B veri akışı yazılır.

7.6 Aynı Veri İçin İki Ok

Her ok tek bir veri akışını gösterir. Bir işlemden iki farklı işleme aynı veri için iki ayrı ok çıkarmak yanıltıcıdır. Veri aynıysa tek bir çizgi çıkar, bir yerde jilet gibi ikiye bölünür ve iki ok olarak yoluna devam eder. Bunu gören kişi tek bir veri akışı olduğunu ve aynı verinin iki işleme de gittiğini temiz bir şekilde anlar.

7.7 Kendine Dönen Akış

Bir veri, hiçbir işlemden geçmeden çıktığı işleme doğrudan dönmemelidir. Bu, rekürsif bir yapı oluşturuyormuş gibi görünür. Normalde veri bir işlemden çıkar, başka bir işlemde işlenir ve işlenmiş bir veri olarak bu işleme tekrar girebilir. Modellemeye göre gerçekten rekürsif bir yapı göstermek gerekiyorsa olabilir, ama onun dışında beklenen bir çizim şekli değildir.

7.8 Akış Üzerinde Fiil

Veri akışlarının üzerinde yalnızca isim formatında yazılar olur, çünkü bunlar veridir, eylem değil. İki örnek:

Yanlış	Sebep	Doğru
Siparişi doğrula	Emirdir, “bir şey yap” der	Doğrulanmış sipariş bilgisi, doğrulanmış sipariş
Zaman çizelgesini tamamla	“Tamamla” bir işlemdir, kutucuk ister	Zaman çizelgesi bilgisi ya da verisi

> **Hocanın vurgusu:** En sık yapılan hata, veri akışına bir işlev gibi fiil yazmaktır. Veri akışlarının hepsi veridir, mutlaka isim olmak zorundadır. Tamamlamak işlemin görevidir.

8. Veri Akış Diyagramı Düzeyleri

Analiz modeli tek bir diyagramla “pat diye” gösterilemez, çünkü sistem katmanlı ve karmaşık bir yapıya sahiptir. Bu yüzden diyagram düzeyler halinde çizilir.

8.1 Taslak (Bağlam) Düzey

İlk dersten hatırlanacağı gibi her sistemin bir amacı vardır. Taslak düzey bunu gösterir. Sistem en soyut seviyede tek bir işlem gibi düşünülür. Sorulan sorular şunlardır: Ana amacım ne, ana sorumluluğum ne, kimden hizmet alıyorum, kime hizmet veriyorum, temel girdilerim ve çıktılarım neler?

Bu en üst seviyeli diyagrama taslak veri akış diyagramı denir. Konteks (context) diyagramı ya da Türkçede bağlam diyagramı diye de duyabilirsiniz. Biri bağlam diyagramını sorarsa aslında taslak düzeyi istiyordur. Özellikleri şunlardır:

- Sistem tek bir kapalı kutudur, kara kutudur.
- Ayrıntılı süreçler yer almaz.
- Veri depoları olmaz, hepsi kutunun içindedir.
- Yalnızca ana varlıklar (etkileşime geçilen taraflar ve dış sistemler), çalışmak için elzem girdiler ve temel çıktılar gösterilir.

Taslak veri akış diyagramı, ön inceleme safhasının sonunda project charter dokümanı yazılırken de oluşturulabilir. Çünkü orada problem ve amaç zaten tanımlanmıştı.

8.2 Alt Düzeyler: Kutunun Kapağını Açmak

Sonraki adım sistemi ayrıntılandırmaktır. Hocanın benzetmesiyle bu, bisküvi kutusunun kapağını açıp içinde neler olduğunu görmek ve resmetmektir. Elde edilen diyagram bir alt seviyedeki veri akış diyagramıdır.

Literatürde iki adlandırma olduğu için karışıklık çıkabilir:

Yaklaşım	En üst	Bir alt	İki alt
Birinci adlandırma	Bağlam (konteks)	Düzey 0	Düzey 1
İkinci adlandırma	Düzey 0 (zemin kat gibi)	Düzey 1	Düzey 2
Bu derste	Taslak, konteks ya da bağlam	1. düzey	2. düzey

Numara mantığı şöyledir: 1. düzeydeki süreçler 1.0, 2.0, 3.0 diye numaralanır. Sonu sıfırla biten bir işlevin kapağı açıldığında 2. düzeye geçilir, örneğin 2.0'ın içindeki işlemler 2.1, 2.2, 2.3 olur.

8.3 Üç Düzey Yeterli

Hoca şu soruyu kendisi sordu: Üç düzey varsa dört, beş olur mu? Hayır. Burada problemi modelliyor, sistemin parçalarını görselleştiriyoruz. Taslak düzey tek kutudur. Ana modüller ve zorunluluklar 1. düzeyde gösterilir. Örneğin fatura hazırlama karmaşık bir işlemse ve alt işlemleri varsa onlar 2. düzeyde gösterilir. "2.1 varsa 2.1.1 niye yok?" sorusunun cevabı da budur: Daha ayrıntılı kısımlar süreç tanım formuyla anlatılır.

> **Hocanın vurgusu:** Taslak, 1. düzey ve 2. düzey olmak üzere üç veri akış diyagramı seviyesi vardır. İkinci düzey, son ayrıntı için bizim açımızdan yeterlidir.

9. Hastane Fatura Sistemi: Taslak ve Birinci Düzey

9.1 Taslak Düzey

Örnek, bir hastane için fatura üretme sistemidir. Esas amaç faturalamadır. Bunun için muayene bilgileri ve sigorta bilgileri alınmalı, fatura hem sigorta varlığına hem hastaya aktarılmalıdır.

```
Doktor --muayene kaydı--> +----------------+ --hasta faturası---> Hasta
                          | Fatura Sistemi |
Hasta  --sigorta formu--> +----------------+ --sigorta faturası-> Sigorta
```

9.2 Birinci Düzey: Dört Ana Sorumluluk

Kutunun kapağı açıldığında dört ana sorumluluk görülür. Veri deposu numaraları kısa nottan alınmıştır.

Süreç	Girdi	Çıktı
1.0 Muayene kaydı inceleme ve ücret hesabı	Muayene kaydı (doktor), ücret (D1 Ücret Listeleri)	Doğrulanmış ücret (2.0'a)

Süreç	Girdi	Çıktı
2.0 Fatura hazırlama	Sigorta formu (hasta), doğrulanmış ücret (1.0), sigorta şirketi adı ve adresi (D3), hasta adresi (D2)	Hasta faturası, sigorta faturası
3.0 Hasta ödemelerini işleme	Hastadan gelen ödeme	Muhasebe kaydı (D4 Muhasebe Kayıtları)
4.0 Sigorta ödemelerini işleme	Sigorta şirketinden gelen ödeme	Muhasebe kaydı (D4 Muhasebe Kayıtları)

> **Ders sonrası not:** Hoca süreçlerin emir kipiyle adlandırılmasını kural olarak koydu, ama bu örnekteki süreçleri "inceleme", "hazırlama", "işleme" gibi isim biçimleriyle okudu. Kurala göre yazım "Fatura hazırla", "Hasta ödemelerini işle" olmalıdır. Slayttaki yazım görülemedi.

Süreç adındaki "ve": 1.0'ın adındaki "ve" şunu anlatır: Bunlar birbiriyle çok yakın ilişkili, bir bütün oluşturan iki eylemdir. Bu nedenle tek işlem olarak modellenmiştir. Büyük ihtimalle 2. düzeye geçildiğinde "muayene kaydı inceleme" ve "ücret hesabı" diye iki alt işlem ortaya çıkacaktır. Sürece daha iyi bir ad da bulunabilir.

1.0'ın işi: Doktor baktı, laboratuvar testleri yapıldı, belki röntgen çekildi. Hepsinin bir maliyeti vardır. Bunlar toplanır ve doğrulanmış bir ücret oluşturulup fatura hazırlama sürecine gönderilir.

> **Ders sonrası not:** Slayttaki "ücret" oku 1.0'dan ücret listelerine doğru çizilmiş. Hoca bunun ters olduğunu iki kez söyledi (00:55:45 ve 00:59:13). Doğrusu okun ücret listelerinden 1.0'a gelmesidir, çünkü ücret listeden okunur.

2.0'ın işi ve iki fatura: Hastadan sigorta formu, sigorta şirketi deposundan başka bilgiler, hasta deposundan hastayla ilgili kayıtlı veriler, 1.0'dan ücret alınır. Burada mesela bir algoritma çalışır ve iki fatura çıkar. Sebebini hoca özel sağlık sigortalarındaki katılım payıyla açıkladı.

Verilen: Muayene 100 lira tutuyor, sigorta %80'ini karşılıyor.

İşlem:

$$\text{Sigorta faturası} = 100 \times 0{,}80 = 80 \text{ lira}$$

$$\text{Hasta faturası (katılım payı)} = 100 - 80 = 20 \text{ lira}$$

Sonuç: İki ayrı bilgi grubu üretilir ve iki ayrı varlığa gönderilir: 20 liralık katılım payı faturası hastaya, 80 liralık fatura sigortaya. Çıktı tarafında hasta varlığı yeniden çizilmiştir, çünkü aksi halde her yerden oklar girip çıkardı. Tekrar edildiğini göstermek için köşesine çentik atılmıştır.

3.0 ve 4.0: Hasta ile sigortanın ödemeleri farklı süreçlerle işlenir, ama sonuçta ikisi de muhasebe kayıtlarında tutulur. Hocanın kaba örneğine göre hasta kredi kartıyla ya da nakit ödüyor olabilir. Süreçler farklıysa 3.1 ve 3.2 diye ayrılır. Sigorta şirketleri böyle ödemez: Aylık cari hesap tutup toplam işlem ve toplam paraya göre mahsuplaşabilir ya da her ay belli bir para verip yıl sonunda artı eksiyi çıkarabilir. Bunların hangisi olduğu probleme bağlıdır. İş analizi tam da bunu ortaya çıkarmak için yapılır.

9.3 Diyagram Kime Ne Söyler?

Esas derdimiz 1. düzey diyagramı oluşturmaktır. Sistemin ana sorumluluklarıyla nasıl çalıştığını, süreçlerin birbirleriyle ve dış kaynaklarla bütün ilişkilerini gösterir.

Yazılımcı için: Sadece veri depolarına bakarak hasta ve sigorta şirketi verilerinin tutulması gerektiği görülür. En azından sigorta şirketi adı ve adresi, hasta adresi, işlem ücretleri, ödeme ve faturalarla ilgili muhasebe kayıtları gerekir. Süreçlere bakınca da modüller belirir: Fatura modülü, iki ayrı ödeme modülü, alt modülleri olan bir muayene-ücret modülü.

Müşteri için: Müşteri diyagrama bakıp “evet, doktordan bu bilgileri alıyoruz, hastadan sigorta formunu alıyoruz, faturayı iki parça üretiyoruz” diyebilir. Eksik bir şeyi de fark edebilir: “Söylemeyi unuttum, yeni çıkan bir kanun nedeniyle kestiğimiz bütün faturaları Sağlık Bakanlığı’nın bir birimine de raporluyoruz.” Analist bu durumda diyagrama “Sağlık Bakanlığı” diye yeni bir varlık ve fatura hazırlamadan çıkan yeni bir veri akışı ekler. Giden veri, hasta ve sigorta faturalarının ikisi ya da yeni bir bileşke fatura olabilir. Müşteri “şimdi oldu” der.

Mimar ve tasarımcı için: Onlar artık başka soruları düşünür: Bakanlığın haberleşme protokolü ve servis yapısı ne? Kimlik doğrulama kullanıcı adı ve parolayla mı, token mekanizmasıyla mı olacak? Bunlar nereden alınacak, geçerlilik süresi ne olacak? Analist bu ayrıntılara girmez, yalnızca problemi modeller. Rapor hazırlama sürecinin içi “bir dünya” olabilir, ama bu düzeyde yalnızca bir işlevdir.

Sistem çok basitse 1. düzey yeterli olabilir.

10. İkinci Düzey ve Düzeyler Arası Denge

10.1 2.0’ın İçi

Fatura hazırlama o kadar basit bir iş değilse, tek sistem kutusunun kapağını açtığımız gibi 2.0 kutusunun da kapağını açabiliriz. Hocanın anlattığı akış şöyledir:

Alt süreç	Girdi	Çıktı
Hastanın sigorta durumunu inceleme	Sigorta formu (hasta), doğrulanmış ücret (1.0)	Sigorta ve ücret bilgileri
Listelenen miktarla faturayı kıyaslama	Sigorta ve ücret bilgileri, sigorta şirketi bilgileri (depo)	Sigorta faturası, hastadan ödenmesi gereken tutar
Hasta faturasını hazırlama	Hastadan ödenmesi gereken tutar, hasta bilgileri (depo)	Hasta faturası

Bu düzeyin kazancı şudur: 1. düzeyde yalnızca fatura hazırlamadan iki faturanın çıktığı görülüyordu ve bu o seviye için yeterliydi. Burada ise hasta faturasını yapmak için önce hastadan ödenecek tutarın bilinmesi, onun için de önce kıyaslamanın yapılması gerektiği görülür.

10.2 Denge Kuralı

Dikkat edilmesi gereken nokta şudur: 2. düzeydeki kutu, 1. düzeydeki 2.0 ile birebir aynıdır. 1. düzeyde 2.0’a ne giriyorsa ve ne çıkıyorsa aynen bu seviyeye taşınır. Yukarıdaki tabloda dışarıdan gelen dört girdi (sigorta formu, doğrulanmış ücret, sigorta şirketi bilgileri, hasta bilgileri) ve dışarıya giden iki çıktı (sigorta faturası, hasta faturası) 2.0’ınkilerle aynıdır. “Sigorta ve ücret bilgileri” ile “hastadan ödenmesi gereken tutar” ise yalnızca 2.0’ın içinde kalan akışlardır (ara adım eklendi).

Aynı kontrol taslak düzey ile 1. düzey arasında da yapılır. Doktordan muayene kaydı, hastadan sigorta formu alınıyor, hastaya ve sigortaya fatura gönderiliyor. Bunlar 1. düzeyde de görünür. Fark, 1. düzeyde veri depolarının da işin içine girmesidir.

> **Hocanın vurgusu:** Bir üst düzeydeki girdi ve çıktılar, varlıklarla olan veri akışları, alt düzeyde korunmalı, aynıları görülmelidir. Alt düzeye geçerken varlıklar, veri depoları, girdiler, çıktılar ve diğer işlemlerle olan bütün ilişkiler aynen taşınır.
>
> **Ders sonrası not:** Hoca taslak düzeyde dört akış saydı (muayene kaydı, sigorta formu, hasta faturası, sigorta faturası). Oysa 3.0 ve 4.0 süreçlerine varlıklardan ödeme girdisi geliyorsa, 1. düzeyde bu dört akışta bulunmayan iki sınır akışı daha vardır. Denge kuralına göre bunların taslak düzeyde de gösterilmesi gerekir. Slayttaki taslak diyagram görülemediği için bu okların orada olup olmadığı altyazıdan doğrulanamıyor.

11. İki Kısa Düzey Örneği

11.1 Varlık Uyumsuzluğu

Taslak düzey şöyle verilmiş: Varlık 1'den A veri akışı sisteme giriyor, sistem Varlık 2'ye B veri akışı veriyor. Alt düzeyde ise şu yapı var: Varlık 3'ten A ve Varlık 4'ten C bir sürece geliyor. Bu süreçten "A türevi" ve "B türevi" diye iki veri akışı başka bir sürece geçiyor. O süreç Varlık 2'ye B akışını gönderiyor.

Hata, seviyeler arasındaki varlık uyumsuzluğudur:

Akış	Taslak düzey	Alt düzey	Durum
A	Varlık 1'den	Varlık 3'ten	Kaynak uyuşmuyor
C	Yok	Varlık 4'ten	Taslakta eksik
B	Varlık 2'ye	Varlık 2'ye	Doğru

Çözüm, modele ve probleme göre neyin doğru olduğuna bağlıdır. Ya taslak düzey ya alt düzey yanlıştır, ya da A hem Varlık 1'den hem Varlık 3'ten okunmalıdır. Örneğin A gerçekten Varlık 1 ve Varlık 3'ten geliyorsa, bu iki kaynak ve Varlık 4'ten gelen C iki düzeyde de gösterilir. Hocanın gösterdiği düzeltmede taslak düzey güncellenmiştir.

11.2 Tek Okta İki Veri

Bir X sürecine, mesela 3. sürece "A ve B" tek okla giriyor. Alt düzeye inildiğinde birinci ve ikinci alt sürece A ve B'nin girdiği görülüyor. Burada hata var mı?

Soru: Bir öğrenci hata olmadığını düşündüğünü söyledi. Hata var mı? **Cevap:** Ne tam evet ne tam hayır. "A ve B" demek çok iyi bir yaklaşım değildir, çünkü her veri akışı bir oktur. A hasta bilgileri, B sigorta bilgileri olsun. "Hasta ve sigorta bilgileri" diye tek ok yazılabilir mi? İkisi tek bir kompozit yapı (bir struct) olarak gönderiliyorsa yazılabilir. Fakat ayrı yapılarsa ve örneğin A bir işlemden, B bir veri deposundan geliyorsa yanlıştır, iki ayrı ok gerekir. Diyagramda okun öncesi çizilmediği için kesin bir şey söylenemez. Tek kaynaktan geliyorsa sorun yoktur.

En temizi, aynı yerden gelse bile A ve B'yi iki ayrı ok olarak göstermektir. Diyagram çok karmaşıklaşırsa "bunları tek yapı haline getirdim" varsayımıyla tek ok çizilebilir. Hocanın özetiyle bu, sorunlu olmaya daha yatkın ama sorunsuz da kabul edilebilecek bir örnektir.

Soru: Bu süreçlerin çıktısının olmaması sorun değil mi? **Cevap:** Tabii ki sorundur, her sürecin mutlaka bir yere bir şey çıkarması gerekir. Bu örnekte yalnızca giren veri akışları tartışıldığı için geri kalanı çizilmemiştir.

11.3 Süreç Modeli ve Veri Modeli

Hoca buraya kadarını özetledi. Süreç tanımı için veri akış diyagramları oluşturuluyor. Diyagramlarda veri depoları, okunan ve gönderilen veriler var, yani süreçlerle birlikte bir de veri modeli gerekiyor. Bu, kavramsal ölçekte bir varlık ilişki diyagramıyla ifade edilir: Veri nesneleri, özellikleri ve aralarındaki ilişkiler. Bu dilime bugün girilmedi. İlerleyen haftalarda varlık ilişki diyagramlarıyla veri modelinin ve tasarımdaki karşılığının nasıl oluşturulduğu görülecek. Kontrol spesifikasyonu ise ders kapsamında işlenmeyecek.

12. Mantıksal ve Fiziksel Veri Akış Diyagramı

Fonksiyonel çözümleme yapıyorsanız veri akış diyagramları sistem analizinizi görsel olarak ifade eden omurgadır. Bu modelleri kullanarak tasarım yapmak istediğinizde yaptığınız temel iş, diyagramlara gerçekleştirme ayrıntılarını eklemektir. Bu yüzden iki kavram konuşulur:

	Mantıksal VAD	Fiziksel VAD
İçerik	Temel işler ve sorumluluklar, varlıklar, akışlardaki veriler, veri depoları	Bunlara ek olarak donanım, yazılım, insan kaynağı gibi ayrıntılar
Safha	Analiz	Tasarımı ifade etmenin yollarından biri
Örnek	Hastane fatura örneği	Depoların veri tabanı tablosu mu dosya mı olduğu açıklanmış hali
Bu dersteki beklenti	Yeterli	Beklenmiyor, bilgi olsun diye anlatıldı

12.1 Eczane Örneği

Hoca, bir eczaneden ilaç almak için mantıksal bir veri akış diyagramı gösterdi. Hastanın ilacı temin edebilmesi için dört ana işlem vardır:

- **İlacı bulma:** Hastadan ilaç bilgileri gelir. Bu işlem ilaç numaralarını ortaya çıkarır.
- **Fiyatları alma:** İlaç numaraları bu işleme gelir, bir veri deposundan fiyat bilgileri alınır. Çıktı "ilaçlar ve fiyatlar" verisidir.
- **Reçete toplamını hesaplama:** İlaçlar ve fiyatlardan miktar hesaplanır.
- **Faturalama:** Miktar bu sürece gelir, hastadan ödeme alınır, fatura üretilir.

"İlaçlar ve fiyatlar" oku, az önceki tartışmanın bir örneğidir: Tek kaynaktan geldiği için tek ok çizilmiştir. Ayrı ayrı "ilaç bilgileri" ve "fiyat bilgileri" olarak da verilebilirdi.

Fiziksel seviyede ise örneğin mantıksal diyagramda hiç geçmeyen "dosya" ve "veri tabanı tablosu" ayrıntıları eklenir, tam olarak ne gelip gittiği açıklanır. Hoca slayttakinin de yeterli ayrıntı olmadığını, ama gerçekleştirme ve yazılım eklemeleriyle zenginleştirildiğinde bir yazılımcının ya da tasarımcının bunu gerçekleştirebileceğini söyledi.

Peki "ilaç ve fiyatlar" akışının ne olduğu nereden bilinecek? Çemberin göbeğindeki veri sözlüğünden. Tasarımcı sözlük girdisini açar ve verinin yapısını, nerede tutulması gerektiğini oradan öğrenir.

13. Süreç Tanımlama Formu

Diyagramlardaki süreçler artık birer kutucuktur. Kullanım senaryolarıyla iş akışlarını yazmıştık, ama bu kutucukların içinde neler olduğuna ve işin nasıl yerine getirildiğine dair diyagram hiçbir şey söylemez. Analiz safhasında süreçlerin iç yapısını da ifade etmek gerekir. Bunun için çok basit bir form kullanılır: Süreç tanımlama formu.

1. ve 2. düzey diyagramlardaki kenarları yuvarlatılmış her kutu için bir form oluşturulur. Formun alanları şunlardır:

- **Numara, ad, tanım:** Sürecin numarası, adı ve genel tanımı.
- **Girdiler:** Girdilerin neler olduğu, diyagramdakinden daha ayrıntılı.
- **Çıktılar:** Birden fazla çıktı veri akışı varsa maddeler halinde.
- **Süreç tipi:** Çevrim içi ve otomatik mi çalışıyor, yoksa istendiğinde elle, örneğin menüden seçilerek mi tetikleniyor?
- **Süreç mantığı:** İşin nasıl aktığı.
- **Çözülmeyen sorunlar:** Analizde açık kalan noktalar.

Süreç mantığını anlatmanın iki temel yöntemi vardır: Sözde kod kullanmak, ya da karar tablosu veya karar ağacı kullanmak. Hoca sonuncuları aslında aynı şey saydığı için üç yöntemi iki başlıkta topladı.

13.1 Eczanedeki Dördüncü Süreç İçin Form

Hoca, eczane diyagramındaki dördüncü süreç için doldurulmuş bir form gösterdi. Süreç mantığı şöyledir:

```
Miktarı oku
Ödemeyi al
IF ödeme > ödenecek miktar
    Para üstü ver
    ödeme = tamam
ELSE IF ödeme = ödenecek miktar
    ödeme = tamam
ELSE
    ödeme = tamam değil
END IF
IF ödeme tamam
    Faturayı üret
ELSE
    Hastaya "ödeme eksik" bilgisi ver
END IF
```

Miktar, formdaki girdi alanında tanımlanan girdiden okunur. Para üstü vermek de ayrı bir iştir, onun da bir fonksiyon gibi kendi süreç tanım formuyla anlatılması gerekir.

Formun kazancı açıktır. Diyagramda hastaya gidip gelen fatura ve ödeme görülür, ama yeterince para alınmazsa ne olacağı görülmez. Bu bilgi ancak formda görülebilir: Ödeme tamamsa fatura üretilip çıkış akışı gönderilir, değilse hastaya bilgi verilir.

13.2 Çözülmeyen Sorunlar

Analiz ve tasarım sırasında açık kalan noktalar bu alana not edilir. Hocanın iki örneği şunlardı:

- **Ödeme biçimi:** Nakit dışında ödeme biçimi var mı, belli değil. Analizde gözden kaçmış ya da müşteriden yeterince alınamamış olabilir, iş analizi gözden geçirilmelidir.
- **Reçete bilgisi:** Bu süreçte ilaç ya da reçete bilgisi yok. Fatura nasıl üretilecek? İlaçların seri kodları ve barkod numaraları, satıldı bilgisinin ilaç takip sistemine girmesi için yazılıyor. Süreç mantığında bu konuda hiçbir şey tanımlanmamış, sözde kodun güncellenmesi gerekebilir.

Bu notların üzerinden tekrar geçilir ve sorunlar ortadan kaldırılmaya çalışılır. Hocaya göre hiçbir zaman tam mükemmeli yapılamaz, ama kafada soru işareti kalmayacak en iyi hal hedeflenir.

14. Sözde Kod

Sözde kod (pseudocode) önceki derslerde görüldüğü için hoca üzerinde çok durmadı. Standart İngilizce kelimelerden oluşan bir alt kümedir ve süreçleri mantıksal olarak ifade etmeyi sağlar. Üç temel yapısı vardır:

- **Sıralı ifadeler:** Art arda gelen komutlar.
- **Kararlar:** `IF`, `ELSE`, `END IF` dallanmaları ya da `SWITCH` / `SELECT CASE` yapıları.
- **Döngüler:** Tekrar eden işler.

Girintili yazı stiliyle yazılır. Dikkat edilecek nokta, satır satır programlama yapılmamasıdır: “Şu değişkene şu değeri ata, modunu al” denmez. “Şunu hesapla, şunu bir yere yaz, şunu oku” gibi genel eylemler yazılır.

> **Ek bilgi (derste anlatılmadı):** Altyazıda bu yöntem için “yapısal değil, yani sözde kod” diye bozuk bir ifade geçiyor (01:21:13). Hoca büyük olasılıkla “yapısal dil” dedi. Standart İngilizce kelimelerden oluşan alt küme tanımı, kaynaklarda büyük olasılıkla “structured English” adıyla geçen gösterime karşılık gelir.

14.1 Stok Kontrolü Örneği

Hoca bir `CASE` örneği gösterdi: İlacın stok miktarı okunur ve duruma göre şu eylemler seçilir.

Stok durumu	Eylem
Kritik stoktan büyük	Hiçbir şey yapma
Kritik stokta	Eczane çalışanına uyarı gönder
Kritik stoktan küçük	Otomatik olarak tekrar sipariş üret
Sıfır (hiç yok)	Muadil ilaçları göster

Bu eylemlerin her biri ayrı bir iştir. Stok miktarı kontrol işleminin içinde eczane çalışanına bildirimde bulunma, otomatik sipariş üretme, muadil ilaç gösterme gibi alt işlemler olmalıdır ki `CASE` içindeki eylemler çalışabilsin. Burada işin ne olduğu, hangi koşulda ne yapılacağı anlatılır. Eylemlerin içinde ne olduğu henüz söylenmez, çünkü tasarım ve algoritmalar bu aşamada yoktur.

> **Ek bilgi (derste anlatılmadı):** Tablodaki son iki satır çakışır, çünkü sıfır stok aynı zamanda kritik stoktan küçüktür. Bu yüzden sıfır durumu ötekinden önce sınanmalı ya da koşul "kritik stoktan küçük ve sıfırdan büyük" diye yazılmalıdır.

15. Karar Tablosu

Normalde bir akış için sözde kod yeterlidir. Fakat çok fazla koşula göre dallanıyorsa ve `IF`, `ELSE IF`, `SELECT CASE` yapıları bir yerden sonra okunamaz hale geliyorsa buna kombinatoryal durum denir. Bu durumlar karar tablosu ya da karar ağacı ile ifade edilebilir. Hocanın ifadesiyle `IF` ile yazılamaz mı? Yazılabilir, ama 50 tane `IF`, `ELSE IF` oluşur. Karar tablosu bunu daha görsel ve anlaşılır hale getirir.

15.1 İlaç İndirimi Örneği

Tablonun üstünde koşullar, altında eylemler, sütunlarda kurallar vardır.

Koşullar	Kural 1	Kural 2	Kural 3	Kural 4
Çalışan hasta	E	E	H	H
Emekli hasta	H	H	E	E
Toplam tutar ≤ 50	E	H	E	H
Toplam tutar > 50	H	E	H	E
Eylemler				
İndirim oranı 1	X			
İndirim oranı 2		X		
İndirim oranı 3			X	
İndirim oranı 4				X
Ek %10 indirim				X

Tablo şöyle okunur: Çalışansa ve tutar 50'den küçük ya da eşitse birinci indirim oranı uygulanır. Kural 4 özellikle ilginçtir: Hasta emekliyse ve ilaç tutarı 50 liradan büyükse hem 4. indirim oranı hem de ek %10 indirim uygulanır. Yani iki farklı eylem arka arkaya çalışır.

15.2 Birbirini Dışlayan Koşullar

Tabloda çalışan satırı evetken emekli satırı hep hayır, çalışan hayırken emekli hep evettir. Buna mutually exclusive (birbirini dışlayan) denir.

> **Ders sonrası not:** Altyazıda bir yerde "mutual, eksplosif" (01:34:36), bir yerde de bozuk bir biçim (01:29:55) geçiyor. Doğru terim, hocanın 01:28:57'de de söylediği "mutually exclusive" ifadesidir.

Bu tablo aslında şunu söyler: Hasta tipi ya çalışandır ya emeklidir. Analiz sırasında başka hasta tipleri de görülmüş olabilir: Gazi, dul, çocuk gibi. O zaman evet ve hayırlar karışır ve iki ayrı satır mantıklıdır. Ama yalnızca bu iki tip varsa tek satır yeter: "Çalışan mı?" evet ya da hayır. Aynısı tutar için de geçerlidir: "Toplam tutar 50'den küçük eşit mi?" tek satırı yeterlidir, çünkü küçük eşit değilse zaten büyüktür.

Ek bilgi (derste anlatılmadı): Dört E/H satırı olduğu için ham kombinasyon sayısı $2^4 = 16$'dır. Birbirini dışlama kuralı nedeniyle bunların 12'si imkânsızdır ve geriye tablodaki $2 \times 2 = 4$ kural kalır. İki satıra indirgendiğinde kombinasyon sayısı $2^2 = 4$ olur, yani tablo hiçbir durum kaybetmeden sadeleşir.

Karar tablosunun da sınırı vardır. Bütün değişkenlere göre bütün durum uzayını $N \times M$'lik bir matris olarak tanımlamak gerekir. Bir süre sonra bu da okunmaz, takip edilemez ve hataya çok açık hale gelir.

15.3 Burs Örneği

Hoca seçim tablosu için ikinci bir örnek olarak burs vermeyi kullandı. Disiplin cezası almışsa öğrenci doğrudan burs alamaz. Almamışsa not ortalaması ve giriş puanı değerlendirilir. Bu örneğin ayrıntısı karar ağacı başlığında verilmiştir.

16. Sonlu Durum Makineleri ve Eş Zamanlı Sistemler

Birbirini takip eden bir olay ya da eylem sırası varsa, yani sistem sıralı ise, sonlu durum makineleri (finite state machine) kullanılır. "Şu koşul varsa şu duruma, öbür koşul varsa bu duruma geç" şeklinde, kurallara göre bir sıralı gidişat kurgulanır.

Bir adım ötesi eş zamanlı sistemlerdir. Burada eş zamanlı çalışan birden fazla sonlu durum makinesi olabilir. Hocanın örneğinde bir yandan hastanın durumu değerlendirilir: Kamu çalışanı mı, sözleşmeli mi, çocuk sayısı 3'ten büyük mü, ilaç tutarı 50'den büyük mü? Bir yandan da ilaç hazırlanırken doktorun yazdığı raporun personel bilgi sistemine "izinlidir" diye girilmesi gerekebilir. Doktor raporu yazdığı anda bambaşka bir sistemde de bir şeyler yapılır. Bunların modellemesi daha karışıktır, durum geçiş diyagramları ve durum diyagramları kullanılır.

Bu ders kapsamında bu ayrıntıya girilmeyecek. Hoca gerekçesini, sistem analiziyle yeni tanışıldığı için konudan soğutmamak olarak açıkladı.

17. Karar Ağacı

Çok karışık `IF` yapıları sözde kodla yazılabilir ama okunmaz hale gelir, o zaman karar tablosu kullanılır. Karar tablosu da bazen zor okunur. Daha görsel bir gösterim karar ağacıdır. Az karmaşık yapılarda ağaç daha görsel olduğu için okumayı ve takip etmeyi kolaylaştırır.

17.1 Burs Ağacı

```
Disiplin cezası aldı mı?
├─ Evet  → Burs yok
└─ Hayır → Not ortalaması 70'ten büyük mü?
           ├─ Evet  → Giriş puanı ortalamadan büyük mü?
           │          ├─ Evet  → yaprak a
           │          └─ Hayır → yaprak b
           └─ Hayır → Giriş puanı ortalamadan büyük mü?
                      ├─ Evet  → yaprak c
                      └─ Hayır → yaprak d
```

Yapraklardaki kararlar yarım burs, tam burs, sadece bir yıl burs ve hiç burs vermemektir. Giriş puanı sorusu iki dalda da tekrar eder. Hoca bunu "aynı şeyi iki kere tekrar ettim maalesef" diye belirtti. Ağacın toplam beş yaprağı vardır: Disiplin dalından bir, diğer dallardan dört (ara adım eklendi).

17.2 İndirim Ağacı

Az önceki indirim tablosu da ağaç olarak çizilebilir. Kökte hasta tipi vardır: Hasta ya çalışandır ya emeklidir. İki dalda da toplam tutar kontrolü tekrar yapılır. Yapraklarda eylemler bulunur.

```
Hasta tipi?
├─ Çalışan → Toplam tutar 50'den büyük mü?
```

```
│            ├─ Hayır → İndirim oranı 1
│            └─ Evet  → İndirim oranı 2
└─ Emekli  → Toplam tutar 50'den büyük mü?
             ├─ Hayır → İndirim oranı 3
             └─ Evet  → İndirim oranı 4 ve ek %10 indirim
```

Ağaçtaki dört yaprak, tablodaki dört kuralın aynısıdır.

Hocanın özetiyle veri akış diyagramları üç seviyede çizilir (taslak, 1. düzey, 2. düzey). Sonra oradaki süreçlerin içini anlatmak için süreç tanımlama formları üretilir. Formdaki süreç mantığı alanı çok karışırsa ya da sözde kodda çok fazla dallanma olursa karar tablosu ya da karar ağacı kullanılır.

18. Veri Sözlüğü

Süreç modellemesinden uzun uzun söz edildi. Veri nesne modeline birkaç hafta sonra eğilinecek, kontrol spesifikasyonuna ise girilmeyecek. Hepsini ortada bağlayan veri sözlüğü, sistemdeki bütün bilişim maddelerini tanımlayan bilgi dağarcığıdır. İçinde “her şey” olur. Her biri için bir sözlük girdisi (entry) oluşturulur:

- Veri akışları
- Veri depoları
- Veri yapıları
- Veri elemanları (atomik elemanlara kadar)
- İşlemler

Veri akışları ve işlemler diyagramlarda ve formlarda kısa tanımlanmıştı, burada ayrıntılı tanımlanır. Bileşik (kompozit) ise öğelerine göre, öğe ise basit verilere göre tanımlar verilir. Bunun birkaç formatı olabilir, ama hepsi metindir. Bir kartela gibi düşünülebilir.

18.1 Veri Akışı Girdisi

Alan	Örnek
Veri akışının adı	Fatura Basma
Tanımı	Akışın kısa açıklaması
Kaynak	1.1 (nereden geliyor)
Hedef	1.2 (nereye gidiyor)
İçerik	Fatura detayları, müşteri detayları (kompozit)
Açıklamalar	Veri akışıyla ilgili ayrıntılar

Veri akışının bir ok olduğu hatırlanırsa kaynak ve hedef, okun iki ucudur. İçerik alanında kompozit yapılar görülür. Elementer bir akış ise örneğin yalnızca bir kod numarası taşır.

18.2 Veri Deposu Girdisi

Alan	Örnek
Veri deposunun adı	Satış Sipariş Form Dosyası
Tanımı	Hangi verilerin, hangi formatta ve özelliklerde saklandığı
Veri yapısı	Satış sipariş kaydı (kendi sözlük girdisi vardır)
Hacim	Günde yaklaşık 100 kayıt
Erişim	Yalnızca sipariş bölümü personeli
Açıklamalar	Ek ayrıntılar

Hoca hacim alanından yararlanmayı bir hesapla gösterdi.

Verilen: Günde yaklaşık 100 kayıt, kayıt başına 1 MB.

İşlem:

$$\text{Ayda} \approx 100 \times 30 = 3\,000 \text{ kayıt}$$

$$\text{Yılda} \approx 100 \times 365 = 36\,500 \text{ kayıt}$$

$$36\,500 \times 1 \text{ MB} = 36\,500 \text{ MB} = 36{,}5 \text{ GB}$$

Sonuç: Bir yılda 36,5 GB veri depolamak gerekir ve tasarım buna göre yapılır.

> **Ders sonrası not:** Hoca hesap sırasında "yılda 12.000 mi?" diye sordu ve hemen kendisi 36.500 diye düzeltti. Doğru sonuç 36.500'dür. Ayda 3.000 değeri 30 günlük ay yuvarlamasıdır; $3\,000 \times 12 = 36\,000$ eder, 500 kayıtlık fark bundan gelir. 36,5 GB sonucu $1 \text{ GB} = 1000 \text{ MB}$ ile elde edilir; 1024 tabanıyla $36\,500/1024 \approx 35{,}64$ olur.

Erişim alanı güvenlik açısından önemlidir: Güvenlikçiler yetkilendirme mekanizmasını buna göre kurar.

18.3 Veri Yapısı Girdisi

Veri deposu girdisinde "satış sipariş kaydı" yapısının tutulduğu söylenmişti. Bu yapının da kendi girdisi vardır:

Veri elemanı	Tür
Müşteri numarası	Elementer
Sipariş numarası	Elementer
Tarih	Kompozit, kendi sözlük girdisi gerekir
Parça numarası	Elementer
Miktar	Elementer
Birim fiyat	Elementer

Slaytta bazı satırların başında yıldız var. Açıklamada, yıldızlı elemanların her parça kaydı için oluşturulması gerektiği yazılıyor.

18.4 Veri Elemanı Girdisi

Elementer her eleman için de bir girdi oluşturulur. Örneğin müşteri numarası:

Alan	Örnek
Ad	Müşteri numarası
Tanım	Müşteriyi tanımlayan numara
Tip	Nümerik
Uzunluk	4 basamak
Aralık	1 ile 6999 arası
Diğer	Varsa diğer ayrıntılar

Hoca bu aralığın neden böyle olduğunu bilmediğini, sistemde ya da müşterinin isteğiyle böyle olabileceğini söyledi: Müşteri "7000 müşterimiz var, hepsine bu numaraları vermek istiyoruz" demiş olabilir.

> **Ders sonrası not:** 1 ile 6999 arasında $6999 - 1 + 1 = 6999$ numara vardır, 7000 değil. Hocanın 7000 sayısı, örnek için söylenmiş yuvarlak bir değerdir. 6999 değeri de 4 basamağa sığar.

Eleman yalnızca belli değerler alıyorsa bir kod tablosu tanımlanır ("şuysa şudur"), yani bir enumerasyon verilebilir.

18.5 İşlem Girdisi

Alan	Örnek
Numara	2.0
Ad	Satış verisini gir
Girdiler	Alınan veri akışları
İşlemler	Süreç tanımında yazılan tanımlar
Çıktılar	Çıkarılan veri akışları

Böylece diyagramda çizilen her şey, içindeki veri yapıları ve onların elementer elemanları sözlüğe girer. İhtiyacı olan herkes gerekli bilgiyi buradan okuyabilir. Siz olmasanız bile kayıtlar üzerinden iş yoluna devam eder.

> **Hocanın vurgusu:** Herkes yapıyor mu? Hayır. Ama bazı projelerde, özellikle askeri projelerde ve savunma sanayiyle ilgili kontratlı işlerde bunların hepsini tek tek yapmak zorunda kalabilirsiniz.

19. Erişilebilirlik Hangi Fizibilitenin Konusu?

Ders sonunda bir öğrenci erişilebilirlikle ilgili soru sordu.

Soru: Sistem analistleri bir sistemi tasarlarken erişilebilirliği, örneğin görme engelli bir bireyin sistemi rahat kullanmasını göz önünde bulunduruyor mu? Bu hangi fizibiliteye giriyor, sosyal mi, teknik mi? **Cevap:** Sosyal fizibiliteye girer. Hedef kitlede görme engelli vatandaşlar ya da fiziksel olarak klavye ve fare kullanamayacak kişiler varsa, sosyal fizibilitede onların gereksinimlerine yönelik bir yapılabilirlik ortaya konmalıdır.

Bu karar zincirleme etki yapar. Teknik fizibilitede bazı şeyler değişebilir. Örneğin yeni bir veri giriş elemanı gerekir: Gözü takip eden bir kamera, klavyede bir yere odaklanılarak o tuşa basıldığını anlayabilir. Analiz modeli de değişir, çünkü o cihazdan gelen verileri girdi olarak alacak yeni modüller gerekir. Bu tamamen probleme ve beklentilere bağlıdır. Müşteri “tamamen görme engelliler için yapıyoruz” derse fizibilite ve analiz ona göre yapılır. “Onlara hizmet etmeyeceğiz” derse böyle bir gereksinim olmaz.

Dersin Toparlaması

Bu derste sistem analizinin ikinci aşaması olan çözümlemeyi fonksiyonel yaklaşımla gördük. Analiz modeli veri nesne tanımı, süreç tanımı ve kontrol tanımından oluşan bir çemberdir, merkezinde veri sözlüğü vardır. Süreç tanımı için veri akış diyagramları çizilir. Dört öğe (varlık, veri akışı, süreç, veri deposu) kurallara uygun birleştirilir: Her süreçte yeterli girdi, işlem ve çıktı bulunur, veri süreçten geçmeden akmaz, oklar tek yönlüdür ve üzerlerinde yalnızca isim yazılır. Diyagramlar taslak, 1. düzey ve 2. düzey olarak ayrıntılandırılır ve düzeyler arasındaki denge korunur. Her sürecin içi süreç tanımlama formuyla anlatılır. Süreç mantığı sözde kodla, karmaşık dallanmalarda karar tablosu ya da karar ağacıyla ifade edilir. Modeldeki her şey veri sözlüğüne girer.

Hocanın çalışma tavsiyesi, veri akış diyagramlarını başka kaynaklardan da araştırmak, örnekleri incelemek ve çizerek el alışkanlığı kazanmaktır, çünkü analiz modeli oluşturulurken bunlara ihtiyaç duyulacak. Gelecek hafta aynı safha nesneye yönelik yaklaşımla ele alınacak: Fonksiyonel çözümlemede işlevleri veri akış diyagramlarıyla gösterdik, nesneye yönelik bir modelleme ortaya koymak istediğimizde bunun nasıl yapılacağı görülecek.

> **Ders sonrası not:** Altyazıda bu kapanışta “DFT’leri çizdim, data flow diyagramları” geçiyor (01:43:38). Kastedilen kısaltma büyük olasılıkla DFD’dir (data flow diagram).

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.